FINANCIAL RISK PLATFORM

WS3 | Frontend Development Plan

Financial Domain Model & Calculation Services

Workstream objective: Provide reusable financial-domain abstractions and a pluggable calculation service layer, then demonstrate breadth through bounded prototype modules.

Product intent: A broad, lightweight, extensible platform foundation. Initial data volumes and module depth are limited deliberately; architecture, control points and expansion interfaces are not treated as throwaway demo code.

> **Synchronization rule: This document is paired with the opposite-side document for the same workstream. Feature IDs, phase IDs, endpoint contracts, statuses and acceptance criteria must remain aligned.**

Shared architecture: Layer 1 Universal Platform Foundation → Layer 2 Configurable Financial Domain → Layer 3 Financial Calculation Modules → Layer 4 Intelligence and Experience. Data governance and security cross-cut all layers.

1. Scope and Feature Coverage

Feature ID	Feature	Scope / implementation intent
FC-01	Canonical instrument model	Common identifiers, balances, currencies, dates, counterparties, cash-flow and classification fields with extensible attributes.
FC-02	Instrument and cash-flow modelling	Instrument-specific terms, schedules, repricing/maturity dates and projected cash flows.
FC-03	Liquidity and LCR/HQLA	Prototype liquidity classification, eligibility, haircuts, outflow/inflow assumptions and metric components.
FC-04	Maturity ladder and liquidity gaps	Time buckets, inflows/outflows and gap summaries.
FC-05	ALM/IRRBB extension points	Rate scenarios, repricing and sensitivity interfaces; full validated model is later work.
FC-06	Collateral and capital extension points	Collateral attributes/haircuts and capital categories/interfaces; production calculations require specialist specifications.
FC-07	Rule and module registry	Versioned rules, input/output contracts, engine versions and module capability metadata.
FC-08	Market-data and assumption interfaces	Versioned assumptions and market-data inputs, with source/provenance; no external feed required for MVP.

2. Phase-Wise Delivery

P0 | Domain discovery and assumptions

Document known entities and provisional assumptions; mark unresolved client fields explicitly.

P1 | Canonical domain model

Define stable core entities plus extension attributes and source mapping boundaries.

P2 | Rule/module registry

Versioned rules, parameter bounds, module registry and input/output contracts.

P3 | Deterministic calculation framework

Common run lifecycle, version binding, intermediate values, warnings and reproducibility.

P4 | Initial module family

Implement bounded LCR/HQLA and maturity-ladder demonstration with synthetic data.

P5 | Expansion interfaces

Specify contracts for ALM/IRRBB, collateral and capital modules without claiming they are fully implemented.

P6 | Domain validation

Golden cases, expert review, edge-case tests and documented calculation limitations.

3. Implementation Responsibilities

Frontend responsibilities

• Build instrument catalogue and record detail views from backend-provided metadata.

• Display schema/module/rule versions selected for each calculation run.

• Provide calculation launch panel and structured results for LCR/HQLA and maturity buckets.

• Show intermediate values, assumptions, warnings and instrument contributions.

• Show available modules versus planned/extension-point modules distinctly; never imply unimplemented models are production-ready.

4. Paired API / Feature Contract Matrix

Endpoint names are proposed contracts for alignment, not a claim that the APIs already exist. Confirm payload schemas in the shared OpenAPI contract before implementation.

Shared feature pair	Capability	Proposed API contract
FE-FC-01 / BE-FC-01	Instrument catalogue/detail	GET /api/v1/instruments; GET /api/v1/instruments/{instrument_id}
FE-FC-02 / BE-FC-02	Module/rule registry	GET /api/v1/calculation-modules; GET /api/v1/rules; GET /api/v1/rules/{rule_id}
FE-FC-03 / BE-FC-03	Calculation run	POST /api/v1/calculation-runs; GET /api/v1/calculation-runs/{run_id}; GET /api/v1/calculation-runs/{run_id}/results
FE-FC-04 / BE-FC-04	Liquidity/maturity outputs	GET /api/v1/calculation-runs/{run_id}/liquidity; GET /api/v1/calculation-runs/{run_id}/maturity-ladder
FE-FC-05 / BE-FC-05	Assumptions/market-data versions	GET /api/v1/assumption-sets; GET /api/v1/assumption-sets/{id}

5. Cross-Cutting Rules

• The canonical model and source mapping are versioned separately. Unknown source attributes are preserved or quarantined by explicit policy, not silently discarded.

• Every calculation run must bind dataset snapshot, schema/mapping version, rule-set version and calculation-engine version.

• Server-side authorization is authoritative. The frontend may hide unavailable actions but does not enforce security.

• No AI-generated code is executed. AI output is validated against schemas and policy, and consequential actions require authorized approval.

• All errors use a structured response with code, message, details and correlation_id. Sensitive payloads and secrets must not be logged.

• Prototype financial modules use synthetic/approved data, documented assumptions and test fixtures; do not claim regulatory certification or Moody's parity.

• The platform is designed for breadth, but production-grade depth is not implied for every financial domain. ALM/IRRBB, collateral, capital, live market data and regulatory feeds require their own specifications and validation.

6. Definition of Done

• Every feature ID in this document has a matching feature ID in its paired frontend/backend document and in the shared feature register.

• Each endpoint has request/response schemas, status enums, authorization expectations and error examples in OpenAPI.

• Frontend components use actual API responses or clearly labelled mocks matching the contract.

• Backend tests cover validation, permissions, version binding and domain logic appropriate to this workstream.

• The cross-workstream integration test passes for the features included in the release.

Change-control rule: Any change to endpoint, field name, enum, versioning behaviour or acceptance criteria must be reflected in both paired documents and the shared contract in the same change.